IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 30 September 2024, accepted 28 November 2024, date of publication 9 December 2024,
date of current version 17 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3513154

RESEARCH ARTICLE

Tuning and Comparison of Optimization Algorithms for the Next Best View Problematic

EVERARDO SHAIN-RUVALCABA AND EFRAÍN LÓPEZ-DAMIAN
Tecnologico de Monterrey, School of Engineering and Sciences, School of Engineering and Sciences, Tecnológico de Monterrey, Monterrey, Nuevo Leon 64849, Mexico

Corresponding author: Efraín López-Damian (eldamian@tec.mx)

This work was supported by Tecnologico de Monterrey for article processing charges.

ABSTRACT The aim of this paper is to tune and compare different optimization algorithms on the Next Best View (NBV) problem, which consists in finding the next position that the sensor or camera needs to take to scan an object or scenery in its totality. A simulated 5 Degree-of-Freedom (DOF) mobile robot with a mounted simulated range sensor was used on a Virtual Reality Modeling Language (VRML) environment, and the space discretization was made using a voxel map. For the objective function, two main factors were included: an area factor to make sure that the image taken by the sensor provides the best possible information, and a motion factor made up of distance and energy sub-factors to reduce the resources used by the robot. Tasks such as a backstepping technique to escape local minima and a dynamic change in the objective function were implemented. The retrievement of the scene was made on an iterative process, and three different optimization methods were tuned and tested: Nelder-Mead, an Evolution Strategy, and Simulated Annealing. A set of experiments comparing the three methods in computational time and retrievement efficiency were made on three different environments with increasing difficulty to test their repeatability, with them being a laboratory model, a room with a cube and a pyramid inside it, and a study room with multiple furniture and windows.

INDEX TERMS Next best view, objective function, optimization, voxel map.

I. INTRODUCTION

Getting true digital representations of a physical object or environment can be a challenge when it comes to autonomous methods. Nowadays, Computer-Aided Design (CAD) technologies are raising on importance among worldwide fabricators, so many objects produced in the industry are brought to reality by converting a 3D digital object to a physical piece. However, there is the opposite situation in which the object exists without a CAD model like the ones made by artisans, or even pieces that indeed were fabricated using digital models or blueprints that were kept by the fabricator, so it is not possible to access them. If we want to obtain the digital object, we need to have some scanning strategy to save time and resources that would be bigger if no strategy was used resulting in a random exhaustive scanning. This strategy is called Next Best View, which is defined as the process to select the view that would provide more information on the object or environment following some criteria, and repeating this process until the scan is complete.

Some applications where this technique can be used are object digitization for reverse engineering such as a core die of the inlet of a diesel engine [1]. It can even be used for non-engineering applications such as criminology providing a reconstruction of a crime scene [2]. A medical approach can also be set in which a whole scene is scanned, although instead of objects or terrain, a set of chronic wounds are reconstructed with the human body being the environment [3]. For industrial applications involving robotics, the NBV is used for environment exploration on autonomous mobile robots required to find and retrieve task-related objects to accomplish their tasks [4], and a more concrete example can be found on industrial facilities in which robots perform an inspection of the facility so that workers can avoid hazardous places when performing

The associate editor coordinating the review of this manuscript and approving it for publication was João Bernardo Ferreira Sequeiros.

activities such as removing toxic substances, and repairing damaged machinery [5].

NBV can be considered as an optimization problem as we seek to find the best view given the available conditions; therefore, two key elements are the objective function and the optimization method used. The objective function sets the conditions and criteria to define how good a view, while the optimization method is an algorithm to reach the best view according to the requirements of the objective function. Both have a huge importance when designing the solution as the first defines the nature of the problem and the second provides the solution.

In this project we will use the NBV approach in order to perform a simulation of modeling an environment such as a the inside of a room, having multiple proposed models each having a different configuration of the room. The sensor will be inside the room and will be able to move freely around the available space, and the goal is to successfully tune the parameters for each optimization algorithm implemented and make comparisons between them. Our two hypotheses are that the changes in the parameters when tuning each optimization method have an impact on their results, and that we can obtain significant differences among our proposed methods.

This paper begins by defining our choosing of the type of robot and dimensions used for the entire work to then define the space discretization method to use. We include a whole section for the objective function with its individual factors: area, motion and quality. We then define and tune by experimenting on a single model the optimization methods which are: Nelder-Mead, an evolution strategy called (1+1)-ES where the "(1+1)" in its name makes reference to the fact that it only produces one offspring from one parent on every iteration and ES is an acronym for Evolution Strategy, and Simulated Annealing, all of them having a backstepping technique to escape local minima among their iterations. Finally, we compare the three tuned methods on three different environments.

The first main contribution of this work is the definition of a new dynamically changing objective function for the NBV problematic adapted to a 5-DOF mobile robot. We implement an area factor which has been used before, but the main novelty in our function is the way we deal with the motion of the robot by dividing it into two factors: distance factor for the 2D movement of the robot on the ground, and an energy factor for the angular and vertical movement of the sensor within the robot. We then contribute on the optimization methods by modifying a previously studied Nelder-Mead with exhaustive search algorithm to now omit the exhaustive search part and keep a pure Nelder-Mead method. We also test the objective function with the (1+1)-ES optimization method and a simulated annealing algorithm, both unused on the NBV problematic to the best of our knowledge. For all of these algorithms we contribute by providing their parameter values after their corresponding tuning to be used on other future similar works. We also develop a backstepping technique to avoid getting trapped at local minima.

Multiple experiments were made on a simulated environment for each individual phase of the whole process presenting great results on all three optimization methods.

II. RELATED WORK

A. ROBOTS

One important element on the Next Best View problem is the type of robot used, as it will determine the degrees of freedom of the problem. The robot proposed in [6] and [7] is a simulated 5 DOF mobile robot with a range sensor, with the first two DOF being the displacement of the mobile base on X and Z axes, a third degree of freedom with a vertical bar that lifts the sensor around the Y axis, and the last two DOF being the pan and tilt angles of the range sensor. This 5 DOF simulated robot has the advantage that it reduces the problem on one degree of freedom by not considering the roll angle, making the optimization process easier and less time-consuming and without compromising a lot the range of possible views, even if the roll angle could provide more views if the horizontal and vertical apertures of the sensor were different as it could use the biggest aperture on any of the pan-tilt angles, it is still the less essential degree of freedom. Another advantage is that this simulated robot could be easily replaced by an actual robot when applying NBV on real world scenarios as most of the mobile robots available that have pan-tilt heads omit roll angle.

We can even have less DOF as proposed on [8] where there are only 4 DOF. In order to reach such DOF, a Cyberware PS cylindrical scanner and a turntable are used. They divide their positional space into two: Positional Space Surface (PSS) and Positional Space Directions (PSD). For the PSS is discretized into cells forming a cylinder and has 2 DOF whereas the PSD is a unique polar system with another 2 DOF adding up to the total of 4 DOF. As it has only 4 variables representing its positional space the computational time of the optimization process would be less than the previous robot and still not compromise the range of possible views due to the configuration of those 4 DOF as they are adapted on a new coordinate system that adapts to those specific DOF. However, we have the disadvantage that it is clearly made for object scanning meaning that the sensor is outside the object and the approach of this project is to scan an environment with the sensor being inside it.

Another approach can be seen on [9] and [10] where 6 DOF manipulators are used. The advantage of these manipulators is that they incorporate all the possible DOF meaning that there are no possible views that are ignored. However, one main disadvantage is that the displacement range of the sensor is limited to the kinematics of the manipulator as due to it being fixed to the ground and only its links have movement as it is intended for object scanning instead of an environment, plus having a more complex path planning that is specific to each model link configuration and dimensions, with [9] using an ABB Industrial Robot 2000 and [10]

using a modular manipulator built of powercube modules by Schunk.

There are more extreme proposals such as the one on [11] where a humanoid robot is used. The first degree of freedom is the position of the free-floating body, the second being its orientation, and the last DOF represent each joint of the humanoid robot. They propose a HRP-2 robot that has 30 DOF (Head: 2 DoF; Arm: 6 DoF x 2; Hand: 1 DoF x 2; Waist: 2 DoF; Leg: 6 DoF x 2) [12], which adds up to a total of 32 DOF for the problem. Although it has the advantage that it can be used in both object and environment recognition as it can move around an object or inside a building, it is clear that it has a big amount of DOF which makes it heavy time-consuming and the path planning is even more complex than the manipulators from [9] and [10]., and even Unmanned Aerial Vehicles (UAVs) according to [2].

Not all mobile robots need to be ground vehicles, we can also have UAVs as proposed on [13], [14], and [15]. This UAVs have 6 DOF being able to move and rotate around all X,Y,Z axes. Their main advantages are their big freedom to move around the environment and depending on their size they could enter small areas that could be almost impossible with a ground vehicle. However, the main disadvantage is their difficulty on path planning as well as their stability control.

B. SPACE DISCRETIZATION

The next important factor to take into consideration is the way in which the workspace is discretized. We understand for discretization the division of a continuous space into a discrete representation, and an easy to understand example is the division of a two-dimensional picture into small pixels. For this specific problem we have a three-dimensional space, meaning that we need three-dimensional discrete elements that are called voxels. There are two main methods to divide the space into voxels: voxel map and octree.

The first approach, the voxel map, is the simplest one. The 3D space is divided into tiny labeled cubes with all of them having the same size which is better called resolution. This approach has been widely used by many authors such as [3], [6], [11], [13], [16], [17], [18], [19], [20], [21], [22], and [23].

An octree is similar to the voxel map in terms of it being composed by labeled cubes. The main difference is that the cubes inside the octree have different resolution within each other, this is done by dividing some of the big cubes into smaller cubes while keeping other big cubes in their original size. Some works that use this octree aproach are [24], [25], [26], and [27]. The work on [28] and [29] uses an octomap which is an octree with probabilistic occupancy estimation. The main advantage of using the octree aproach instead of a voxel map is that we are more protected against imperfect sensor readings as it increases the resolution on specific areas of the 3D space.

A hybrid method is proposed on [30] by initially using a voxel map, and after labeling all of the voxels only the ones that are occupied will have their resolution increased using octrees. They refer to this approach as semioctree and its advantage is that it increases the model resolution while still keeping it homogeneous.

These proposed discretization methods are three-dimensional although we can have 2D discretization as shown on [31]. They do not divide the 3D space into any discrete element, instead they perform the discretization after taking the 2D image of the object and divide it into pixels. Then, they build local feature histograms with probabilistic recognition based on the available photos. This has the inconvenient that a data base containing huge amounts of views of multiple objects is needed, making it use a lot of resources and forcing a prior knowledge of the model making it less generalized.

C. OBJECTIVE FUNCTION

The first step of the optimization process is the declaration of an objective function which will be used for setting up a criteria for deciding which view is better, and this function can have multiple factors.

The most essential is the area factor that analyzes the proportions of the view depending on the labels of the voxels from the voxel map or octree. It is possible to have the objective function with only this area factor as shown on [6].

Another factor that can be used is the quality one. The work by [6] besides from showing a case where only the area factor was used, it also included a second scenario where they add a quality factor. The proposal by [18] has also both factors although they refer to the area factor as visibility factor but is equivalent. The advantage of having this quality factor is that we are more protected from misleading sensing measurements as the rays from the sensor are more perpendicular to the surface although it can considerably increase the number of views.

Other works add an extra factor for distance. The objective function of [7] uses one factor that combines area and quality into a single equation and adds the distance factor considering 3D Euclidean distance. A similar approach can be seen on [32] where they use area, quality and distance factors. However, instead of using 3D Euclidean distance they use the normalized orthodromic distance. Finally, on [28] they consider an area factor, a distance factor using Euclidean distance and a positioning factor to make sure that the space is free for the robot to be there. The advantage from the distance factor is that it optimizes the movement of the robot which reduces the amount of power needed and from an economic point of view it saves resources.

D. OPTIMIZATION METHOD

Optimization method refers to the algorithm used for finding the best candidate view based on the objective function. Testing all the possible views would ensure that the absolute best is selected. However, this would be extremely time-consuming.

The work by [33] compares two Random-Sampling-Based Next-Best View Exploration (RNE) algorithms which propose random views systematically.

- The first is the Rapidly-Exploring Random Tree (RRT) algorithm in which random samples are generated according to a tree and nodes structure. According to the authors, this first method was inspired by the Receding Horizon- Next Best View Planing (RH-NBVP) algorithm proposed by [15] which is also used on [34] and further improved on [35] due to its high computation time. This RRT algorithm is also used on [36], where a two-phase approach is proposed with the first one being the exploration stage using RRT and the second being a relocation stage.
- The second algorithm on their comparison is Rapidly-Exploring Random Graph (RRG) which takes as a base the RRT algorithm but replaces the tree with a graph allowing an advanced connectivity between the nodes as well as making it asymptotically optimal according to [37]. This RRG algorithm is also used on [38].

Another approach is using the Nelder-Mead simplex algorithm as shown on [39]. This algorithm builds an initial simplex with one more dimension than the number of variables and manipulates the simplex until a stopping criteria is fulfilled. The work by [6] performs a two-phase optimization method, with the first being for the position of the robot using this Nelder-Mead simplex algorithm and the second being an exhaustive search on the pan-tilt space for the orientation of the sensor. The advantage of this approach over the RNE algorithms is that only the initial simplex is random, all the other steps do not include any type of randomization making it more repeatable.

A regression approach can be seen on [23] where they use a convolutional neural network algorithm to perform the regression for the position of the robot whereas the orientation is computed using geometric reasoning. The main disadvantage of this approach is that a training for the convolutional neural network is needed and there may be the chance that such training needs to change according to the environment or objects used.

III. INITIAL CONSIDERATIONS

A. ROBOT

According to the information presented on the related work section, the most adequate robots for environment scanning are UAVs, humanoid and 5 DOF mobile robots. We rule out the consideration of using a humanoid robot due to its high number of DOF which unnecessarily increases the difficulty and time-consumption of the problem as we do not have the need of using such robot as it was the case on [11], we are interested in navigation for mobile robots.

From the remaining proposed options, we will choose a 5 DOF mobile robot instead of UAVs as it reduces the problem on one non-essential degree of freedom as well as having an easiest control of its movement if the physical implementation is needed on future work. As we mentioned before, this 5 DOF mobile robot configuration is not exclusive to any brand and can be expressed as a general model which could even be home-made.

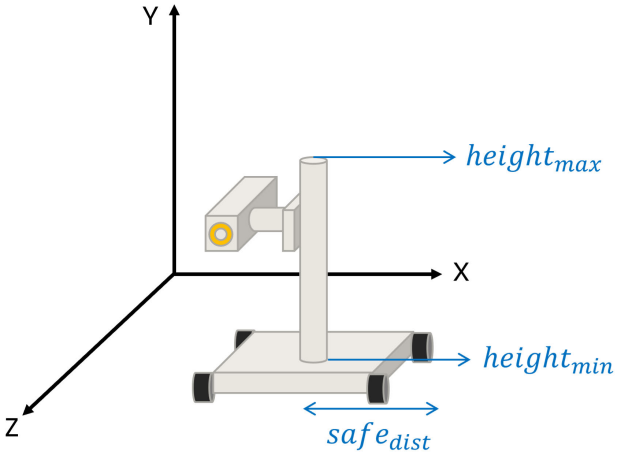


FIGURE 1. Robot dimensions.

1) ROBOT DIMENSIONS

Although it may seen trivial, the robot dimensions are crucial to its behaviour. As we are dealing with recovery of an environment, the robot used should be suitable for the given conditions. For example, if we are trying to retrieve a room which has a height of 2 meters, the robot should not exceed that height as for obvious reasons it would not even fit inside the room.

Even if we are working with different models of robots with different complexity, for our simulation we just need to indicate three dimensions which will generalize the robot. This three dimensions are:

- Minimum sensor height $height_{min}$
- Maximum sensor height $height_{max}$
- Sensor safe distance $safe_{dist}$

A graphical representation of these three measurements with the proposed general configuration of the robot can be seen on Fig. 1, where we notice that the minimum sensor height is the lowest it can move and maximum sensor height is the highest point it can get, whereas the sensor safe distance represents half of the robot width, and as we have only one measurement variable but two axis, it will correspond to the highest of the X or Z axes robot widths. One recommendation is to exaggerate a little this sensor safe distance by increasing it, as it will be used to form a cylinder around the robot used for collision checking, so it is better to have a tolerance around the robot by increasing this value. This sensor safe cylinder will be able to be turned on or off during the simulation, and is represented on Fig. 2, where we can see that it has a radius equal to the sensor safe distance, and a height equal to the maximum sensor height without taking into consideration the minimum sensor height as it goes from

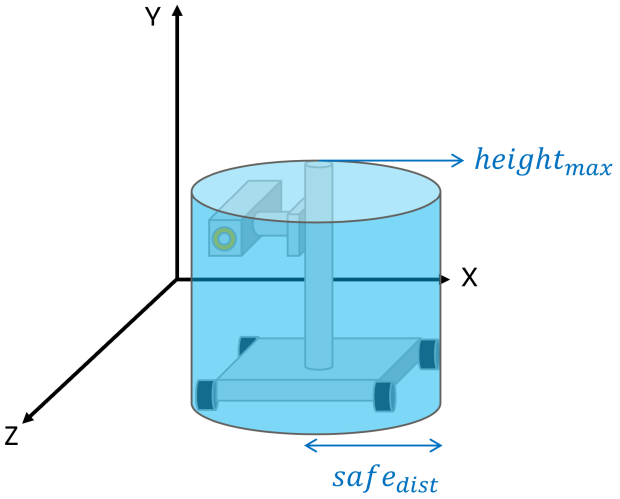


FIGURE 2. Safe cylinder.

the ground because the base also needs to be considered inside the cylinder.

2) IMPLEMENTATION OF ORIENTATION ICOSAHEDRON

Although we have 3 DOF for traslation and 2 DOF for orientation adding up to a total of 5, this only represent the physical DOF of the robot. If we made no further modification then the DOF of the optimization model would also be 5. However, we can add an orientation icosahedron as shown on [6] and [32]. This reduces the pan-tilt angles into just one variable v that goes from 0 to 19 and represents the face of the orientation icosahedron with each of them having an associated set of pan-tilt angles; therefore, we reduce one degree of freedom of the optimization model and end up with a total of 4 DOF. The ray of the sensor on the face v of the icosahedron is projected on the normal vector of that face, with the list of pan-tilt angles per each value of v being presented on Table 1.

3) SENSOR

The sensor used will be a simulated range sensor developed by [6] and used previously on [6], [7], [20], [28], [29], [32], and [23]. It uses a 3D Bresenham algorithm [40] with also implementing the ARTS method from [41] to simulate ray tracing, and we can see that this type of sensor is widely used because it is a general representation and its characteristics can be assigned through parameters during simulation.

The first parameters are related to the range in which the sensor can detect the surface, and those are the maximum and minimum distance. We need to consider that the surface normal will not always be parallel to the ray of the sensor, there can be an inclination between those two so we define a maximum sensor view angle. Next, we have the parameters corresponding to the section of the surface that can be sensed. Horizontal and vertical aperture angles are defined and this are projected creating a 2D image which is compounded of

TABLE 1. Pan-tilt angles of orientation icosahedron.

Icosahedron face v	Pan angle (degrees)	Tilt angle (degrees)
0	0	69.094843
1	-45	54.735611
2	-90	20.905158
3	90	20.905158
4	45	54.735611
5	69.094843	90
6	110.905158	90
7	135	54.735611
8	180	69.094843
9	180	110.905158
10	135	125.264391
11	90	159.094844
12	-90	159.094844
13	-45	125.264391
14	0	110.905158
15	45	125.264391
16	-69.094843	90
17	-110.905158	90
18	-135	54.735611
19	-135	125.264391

points and has horizontal and vertical dimensions. Finally, sensors in real life have slight errors that need to be considered on this simulated sensor as it is intended to imitate a real sensor. To cause this slight imperfections we add Gaussian noise with a mean of 0 and a standard deviation of 1, with an amplitude of our last parameter that represents the sensor noise deviation.

As a summary, all the sensor parameters are:

- Sensor view angle
- Minimum sensor distance
- Maximum sensor distance
- Sensor noise deviation
- Sensor horizontal aperture
- Sensor vertical aperture
- Sensor horizontal points
- Sensor vertical points

B. SPACE DISCRETIZATION

The two main 3D space discretization methods presented on the related work section were voxel map and octree. The main reason of using an octree was to reduce the probability of a misreading from the sensor, but as we are working in a simulation where we do not have the imperfections from the actual sensor there is no need to upgrade from voxel map to octree; therefore, we will use voxel map for space discretization.

1) VOXEL MAP LABELING

The traced rays of the sensor will pass through the voxels on its path and will determine their type according to the following classification:

- Unmarked voxel ($V_{unmarked}$): a voxel that has not yet been explored by the ray tracing, it is unknown if it has points of the environment inside it or not.

- Empty voxel (V_{empty}): a voxel that has been explored by the ray tracing and has no points of the environment inside it making it empty.
- Occupied voxel ($V_{occupied}$): a voxel that has been explored by the ray tracing, has points of the environment inside and is not adjacent to an empty voxel.
- Border voxel (V_{border}): a voxel that has been explored by the ray tracing, has points of the environment inside and is adjacent to an empty voxel.
- Occluded voxel ($V_{occluded}$): a voxel that is not adjacent to an empty voxel and is occluded by an occupied or border voxel making it unfeasible for the ray tracing, it is unknown if it has points of the environment inside it or not.
- Occplane voxel ($V_{occplane}$): a voxel that is adjacent to an empty voxel and is occluded by an occupied or border voxel making it unfeasible for the ray tracing, it is unknown if it has points of the environment inside it or not.

The voxel map VM will have vx, vy and vz as indexes for all its voxels, and its update will be done after each view is taken.

IV. OBJECTIVE FUNCTION

An objective function is a mathematical expression that is dependent on variables that characterize a phenomenon, and that its value depends on the desired behavior indicated when designing the function, with aims to have the highest result at the most desired values of its variables. Our objective function f will contain an area factor f_{area} as its principal factor, and the secondary ones will be distance factor f_{dist} and energy factor f_{energ}, with f_{dist} having more priority than f_{energ} as shown on Equation 1.

$$f = f_{area}(0.7 f_{dist} + 0.3 f_{energ}) \tag{1}$$

A. AREA FACTOR

The function for this factor can be seen as:

$$f_{area1} = \left(\frac{2a_{ov}^3 - 3\,(\alpha+1)\,a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3}\right) \cdot \left(1 - 0.5\left|a_{op} - a_{us}\right|\right), \alpha = 0.4 \tag{2}$$

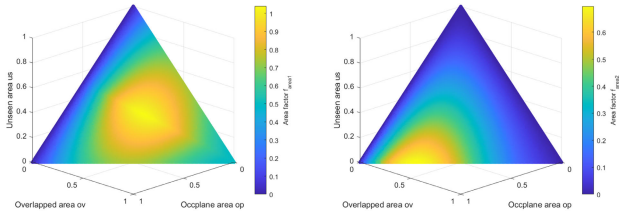
where a_{ov} is the proportion of overlapped area, a_{op} the proportion of occplane area, a_{us} the proportion of unseen area, and α the optimal proportion of overlapped area. With a_{ov}, a_{op} and a_{us} being calculated as:

$$a_{ov} = \frac{n_{occupiednotborder} + n_{border}}{n_{total}} \tag{3}$$

$$a_{op} = \frac{n_{occplane}}{n_{total}} \tag{4}$$

$$a_{us} = \frac{n_{unmarked} + n_{occluded}}{n_{total}} \tag{5}$$

where n_{total} represents the total number of voxels in the current image, $n_{occupiednotborder}$ the number of occupied voxels, n_{border} the number of border voxels, $n_{occplane}$ the number of occplane voxels, $n_{unmarked}$ the number of unmarked voxels and $n_{occluded}$ the number of occluded voxels.



(a) Evaluation of first area factor (b) Evaluation of second area factor

FIGURE 3. Area factor functions.

This function aims to provide overlap with previously acquired views for fine registration of the data, eliminate occlusion plane areas, observe new unseen areas and favour at the same time the sensing of occlusion plane and unseen areas as stated on [6]. It was developed by taking the original function $f_{\text{area}} = \left(5a_{ov}^3 - 10.5\,a_{ov}^2 + 6a_{ov}\right)(1 - 0.5\left|a_{op} - a_{us}\right|)$ from [6] and transforming it into a general expression to have α as a parameter, which after multiple experiments outside of the scope of this paper we determined that the best value was 0.4, and to normalize the function so that the maximum is 1 instead of 1.04 which was the maximum of the original function.

We will apply a dynamic change in this factor when the percentage of unmarked voxels in the entire voxel map is less than 1%, where the new function will replace the term $\left(1 - 0.5\left|a_{op} - a_{us}\right|\right)$ with just a_{op} shown on Equation 6 as at this point the unseen area should be irrelevant to the function.

$$f_{area2} = \left(\frac{2a_{ov}^3 - 3\,(\alpha+1)\,a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3}\right)(a_{op}), \quad \alpha = 0.4 \tag{6}$$

making the final area factor as:

$$f_{area} = \begin{cases} f_{area1}, & \dfrac{n_{unseen}}{n_{total}} > 0.01 \\ f_{area2}, & \dfrac{n_{unseen}}{n_{total}} \leq 0.01 \end{cases} \tag{7}$$

where n_{unseen} is the number of unmarked voxels and n_{total} is the total voxels on the map.

We can plot both resulting functions on a three-dimensional plot shown on Fig. 3, where we represent the result of the area factor as a fourth dimension labeled by colors, where their maximum is located at the yellowest coordinate on both cases and we notice that the maximum of f_{area2} is lower on the axis corresponding to unseen area proportion as it does not consider it inside the function.

B. SECONDARY FACTORS

If we only considered the area factor proposed on the previous section, it will be expected to have the best performance regarding the recovered percentage of the scene as it would have just one priority being the voxels information of the picture. However, if we intend to implement this on a real

robot we will not have infinite resources, that is why extra factors are needed.

1) DISTANCE FACTOR

This distance factor gives priority to views in which the displacement of the base of the robot is smaller. First we need to have a measurable variable that will be used inside the factor itself, and will be calculated as a 2D euclidean distance d:

$$d = \sqrt{d_x^2 + d_z^2} \tag{8}$$

where d_x is the distance around the X axis and d_z is the distance around the Z axis, both of them are expressed in meters as well as the main distance d. Distances around each axis need to be calculated as they are not explicit inside the program, it can be done by subtracting the current position to the NBV position:

$$\begin{aligned} d_x &= NBV_x - Sensor_x \\ d_z &= NBV_z - Sensor_z \end{aligned} \tag{9}$$

where NBV_x is the X axis position from the next best view, NBV_z the Z axis position from the next best view, $Sensor_x$ the current X axis position from the sensor and $Sensor_z$ the current Z axis position from the sensor

Finally, once we have this distance, it needs to be normalized because we need to have our objective function between 0 and 1 always, and for that to happen each factor needs to be on the same range, so we need a maximum distance and a minimum distance. The minimum distance should be 0, as there can be occasions in which the base of the robot does not move and is valid as the sensor can still move around the robot itself. However, the maximum distance is hard to define, as not all the environments have the same size. One solution that was considered was to read the parameters from the file of the desired model and obtain a maximum distance d_{maxEnv} regarding the environment. However, this proposal had a main disadvantage as it could not be implemented on a real scenario in which we will not have the measurements of the scene in advance. For this reasons, it was decided that the maximum distance needs to be a constant value d_{max} expressed in meters regardless of the environment size, and it is considered that $d_{max} = 10m$ is a great value that adjusts to a great range of environment sizes without affecting its impact on the objective function. The normalized distance d_{norm} will be dimensionless, and be computed as:

$$d_{norm} = \frac{d}{d_{max}} \tag{10}$$

We can use as a base the work of [32] and adapt it as it was made considering the normalized orthodromic distance but we will use our distance d_{norm}. The main characteristics defined by them are:

1) The candidate view with the shortest distance must have the maximum value
2) The candidate view with the largest distance must have a minimum value given by ρ
3) A candidate view with a larger distance than other must have a smaller value.
4) At short distances, the value of the function must decrease more slowly than at large distances.

The first three constraints seem to work with our desired behaviour, so we will keep them. However, the last constraint would cause that distances that are not big enough would not have an impact on the function. To deal with this, we can propose a linear behaviour instead:

$$f_{dist} = \begin{cases} (\rho_{dist} - 1)d_{norm} + 1, & d_{norm} \leq 1 \\ 0, & d_{norm} > 1 \end{cases}, \quad \rho_{dist} = 0.1 \tag{11}$$

2) ENERGY FACTOR

This energy factor gives priority to views in which the movement of the sensor around the robot itself is smaller. First we need to have a measurable variable that will be used inside the factor itself. This variable needs to include the distance around Y axis d_y expressed in meters, as well as the pan angle ψ which is the angle around the Y axis and tilt angle θ which is the angle around X axis, both expressed in degrees. We can propose a symbolic variable that we will call "energy" represented by the term e, and although the term energy is already an existing quantitative property on physics, we will use it to describe the movement of our sensor around the robot, as this movement is not a concrete distance but still consumes energy from the robot, whether it is electrical, pneumatic or hydraulic depending on the robot.

First, we calculate the distance on the Y axis d_y, the pan angle ψ and the tilt angle θ:

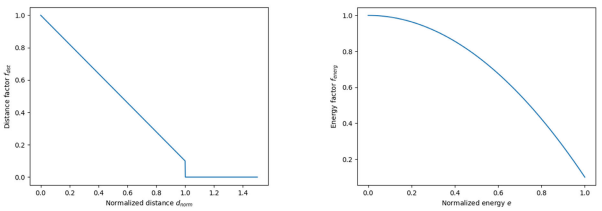
$$\begin{aligned} d_y &= NBV_y - Sensor_y \\ \psi &= NBV_\psi - Sensor_\psi \\ \theta &= NBV_\theta - Sensor_\theta \end{aligned} \tag{12}$$

where NBV_y is the Y axis position from the next best view, NBV_ψ the pan angle from the next best view, NBV_θ the tilt angle from the next best view, $Sensor_y$ the current Y axis position from the sensor, $Sensor_\psi$ the current pan angle from the sensor and $Sensor_\theta$ the current tilt angle from the sensor.

We proceed to compute the normalized Y axis distance dy_{norm}, the normalized pan angle ψ_{norm} and normalized til angle θ_{norm} so they are dimensionless and can be used on a single expression:

$$\begin{aligned} dy_{norm} &= \frac{d_y}{height_{max}} \\ \psi_{norm} &= |\frac{\psi}{180°}| \\ \theta_{norm} &= |\frac{\theta}{180°}| \end{aligned} \tag{13}$$

Once we have each normalized variable, we can create our energy e. We can make an average of the three variables as



(a) Evaluation of distance factor (b) Evaluation of energy factor

FIGURE 4. Secondary factors.

all of them are dimensionless and have the same importance.

$$e = \frac{dy_{norm} + \psi_{norm} + \theta_{norm}}{3} \tag{14}$$

Now we need to define the function for the energy factor. We will use as a base Equation 11 but we will keep the second-degree polynomial proposed by [32] for bigger decrease at higher energy values:

$$f_{energ} = (\rho_{energ} - 1)e^2 + 1, \quad \rho_{energ} = 0.1 \tag{15}$$

On Fig. 4 we can see both distance and energy factors, where the difference aside from the variables used is the linearity of Fig. 4(a) against the second-degree polynomial shape of Fig. 4(b), plus we also consider a higher domain on the X axis on Fig. 4(a) as d_{norm} can have values higher than 1 where the evaluation of the function will be 0.

V. OPTIMIZATION

We already have our objective function on Equation 1 with its area factor from Equation 7, distance factor from Equation 11, and energy factor from Equation 15. We also have the decision variables which will are:

- Position: NBV_x, NBV_y, NBV_z.
- Orientation: v.

The feasible space will be for those integer values of v between 0 and 19 and for those values of NBV_x, NBV_y, and NBV_z in which the sensor falls inside the map and the safe cylinder is full of only empty voxels. As this will be an iterative process, we will apply the optimization methods within each iteration, and this can result in an scenario in which the decision variables have the same values for two consecutive iterations causing the robot to get stuck on local minima. For this, we define a backstepping technique to artificially move the robot on those situations. To trigger this backstepping, the robot must not move between two iterations (both e and d_{norm} need to be 0) and the total percentage retrieved from the original model need to be less than 100%, meaning that we need to keep improving the retrieved model. Once those conditions are met, the backstepping will be triggered and will return the robot to the latest position that is at least $3m$ away from the current position. Note that each position that the robot is returned to via backstepping is added to a register so that it is avoided on future iterations that also need backstepping, the only possibility for this register to be wiped out is when the area factor from the objective function is changed from f_{area1} to f_{area2}. Having defined this task, we can now define all of the optimization methods that will be used by following some classifications.

First, we have the conventional methods which are direct search and gradient-based [42]. We do not want to have gradient-based methods as we do not have a clear mathematical relationship between the variables from the sensor and the ones used on the objective function as it is done through sensing simulation, we need methods that only use the evaluation of the objective function, making direct search the best approach. The Nelder-Mead algorithm used by [6] and [7] developed by [43] corresponds to local search [44], [45], [46], it will be our first proposed method.

Aside from conventional methods, we have metaheuristics which are methods that try to imitate nature behaviour. A general classification of this metaheuristics is presented on [47] where the main criterion is the number of initial points considered. Single-solution based metaheuristics use one initial point and move away from it trying to upgrade the solution; on the other hand, population-based metaheuristics build a set of initial points. As the already proposed Nelder-Mead algorithm has an initial simplex built with multiple points, we will have a bigger focus on methods with a single initial point to have bigger difference among the proposed methods.

Even if evolutionary algorithms are considered population-based metaheuristics, there is one method called (1+1)-ES that uses a single initial point as a parent and produces a single offspring on each iteration [48]. We will add this algorithm to our list as it has a single initial point and follows a random strategy contrary to the local search of the previous method. Lastly, we can add a different approach on our set of methods by using a simulated annealing annealing which is a probabilistic method.

Below we can see all three proposed methods with their main characteristics:

1) Nelder-Mead algorithm:
   - Conventional method
   - Local search
2) (1+1)-ES:
   - Metaheuristic
   - Random strategy
3) Simulated annealing:
   - Metaheuristic
   - Probabilistic method

We will treat all of this methods as minimizations, therefore we will multiply by -1 our objective function to turn it into a minimization as it is originally a maximization, and we will assign a value of infinity to the evaluations in which the constraints were not fulfilled.

A. NELDER-MEAD ALGORITHM

This method creates an initial simplex with one more dimension than the number of variables used, and performs certain operations (reflection, expansion, contraction and

reduction) on the simplex until it reaches a tolerance value indicating that it reached an optimal value. The steps of Algorithm 1 are based on the code provided on [49] with some additional adaptations.

On this method we will not have to tune any parameters although we will still make a comparison between two versions: one with exhaustive search as proposed on [6] and one without it as we believe it consumes a lot of computational time. When the exhaustive search is used we will evaluate the proposed points with a function *exh*() that will evaluate the objective function on all 20 possible values of v on the specified position and return the best of them, whereas without the exhaustive search the evaluation of each point will be made by just evaluating the objective function $f()$ on the specified position and orientation.

B. (1+1)-ES

This optimization algorithm is based on the nature of evolution. It takes an initial parent p_{parent} that produces an offspring $p_{offspring}$ slightly different than its parent by a mutation using the strength of mutation σ and a random number $random_{gauss}$ following a Gaussian distribution with mean 0 and standard deviation of 1. For the parameters, we will use initially 500 on gen_{max} as it is a big number and although it may increase the computational time, we will later analyze the results and develop a method to obtain a smaller gen_{max} making it a variable parameter. We will fix c to 0.817 as it has been proven to be an adequate value for the algorithm regardless of its application [50] an has been developed theoretically on [51]. Finally, the initial value $\sigma_{initial}$ of σ will be made variable to be later tasted with multiple values. The steps of Algorithm 2 are based on the algorithm from [52] and adapted to this specific problem.

C. SIMULATED ANNEALING

This algorithm is different as it sometimes accepts bad solutions as the new point giving it the opportunity to explore new areas by providing a high probability for the first iterations and decreasing it for bigger iteration values. It starts by setting the current coordinates of the sensor as a reference point p_{ref} and then by creating a neighbour point p_{new} close to p_{ref}, and it replaces it if it is a better point or if an acceptance probability related to the temperature T_k is reached, to finally cool T_k using a cooling ratio $cool_{ratio}$ every certain interactions. We can tune the initial temperature T_0 by defining our desired initial probability of acceptance as 95% for the worst case scenario that represents the biggest possible change in the function evaluations which for our problem is 1. This gives us the analysis of Equation 16 where we compute the value $T_0 = 19.5$.

$$\begin{aligned} 0.95 &= e^{\frac{-1}{T_0}} \\ T_0 &= \frac{-1}{ln(0.95)} \\ T_0 &= 19.5 \end{aligned} \tag{16}$$

Algorithm 1 Nelder-Mead Algorithm

Input: Current sensor coordinates
Output: Optimal point p_{opt} for NBV
if *exhaustive search is used* **then**
  $ndim \leftarrow 3$; `// # of variables`
  $eval() \leftarrow exh()$; `// Function to evaluate points`
else
  $ndim \leftarrow 4$;
  $eval() \leftarrow f()$;
end
`/* Build the initial simplex` p `with points` $p_1, \ldots, p_{ndim+1}$ `*/`
$p_1 \leftarrow$ current sensor coordinates;
$p_2, \ldots, p_{ndim+1} \leftarrow$ random feasible coordinates;
$nelder_{iter} \leftarrow 0$; `// Iterations`
$nelder_{range} \leftarrow 1000$; `// Tolerance`
while $nelder_{iter} < 1500 and nelder_{range} \geq 0.0001$ **do**
  Order simplex points so that $eval(p_1) \leq eval(p_2) \leq \ldots \leq eval(p_{ndim}) \leq eval(p_{ndim+1})$;
  $p_0 \leftarrow$ centroid of $p_1, \ldots, p_{ndim}$; `// Centroid`
  $p_r \leftarrow p_0 - (p_{ndim+1} - p_0)$; `// Reflected point`
  if $eval(p_1) \leq eval(p_r) < eval(p_{ndim})$ **then**
    $p_{ndim+1} \leftarrow p_r$;
  else
    if $eval(p_r) < eval(p_1)$ **then**
      $p_e \leftarrow p_0 - 2(p_{ndim+1} - p_0)$;
      `// Expanded point`
      if $eval(p_e) < eval(p_r)$ **then**
        $p_{ndim+1} \leftarrow p_e$;
      else
        $p_{ndim+1} \leftarrow p_r$;
      end
    else
      $p_c \leftarrow p_0 + 0.5(p_{ndim+1} - p_0)$;
      `// Contracted point`
      if $eval(p_c) < eval(p_{ndim+1})$ **then**
        $p_{ndim+1} \leftarrow p_c$;
      else
        for $i = 2 : ndim - 1$ **do**
          $p_i \leftarrow p_1 + 0.5(p_i - p_1)$;
        end
      end
    end
  end
  $nelder_{range} \leftarrow \frac{2|p_{ndim+1} - p_1|}{|p_{ndim+1}| + |p_1|}$;
  $nelder_{iter}++$;
end
$p_{opt} \leftarrow p_1$;

We tune $cool_{ratio}$ by defining our desired final probability of acceptance as 5% for the worst case scenario as well. This gives us the analysis of Equation 17 where we obtain the value $cool_{ratio} = 0.85$ and we confirm that it is a feasible value as

Algorithm 2 (1+1)-ES Algorithm

Input: Current sensor coordinates, gen_{max}, $\sigma_{initial}$
Output: Optimal point p_{opt} for NBV
$c \leftarrow 0.817$; // Parameter
$m \leftarrow 0$; // Successful mutations
$\sigma \leftarrow sigma_{initial}$;
$p_{parent} \leftarrow$ current sensor coordinates;
for $gen = 1 : gen_{max}$ **do**
  /* Mutation */
  for $i = 1 : 4$ **do**
    $p_{offspring}[i] \leftarrow p_{parent}[i] + \sigma \cdot random_{gauss}$;
  end
  /* Selection */
  if $f(p_{offspring}) < f(p_{parent})$ **then**
    $p_{parent} \leftarrow p_{offspring}$;
    $m++$;
  end
  /* σ update */
  if $g\%20 == 0$ **then**
    if $\frac{m}{gen} > 0.2$ **then**
      $\sigma \leftarrow \frac{\sigma}{c}$;
    else if $\frac{m}{gen} < 0.2$ **then**
      $\sigma = \sigma \cdot c$;
    end
  end
end
$p_{opt} \leftarrow p_{parent}$;

Algorithm 3 Simulated Annealing

Input: Current sensor coordinates, L
Output: Optimal point p_{opt} for NBV
$cool_{ratio} \leftarrow 0.85$;
$T_0 \leftarrow 19.5$;
$T_k \leftarrow T_0$;
$p_{ref} \leftarrow$ current sensor coordinates;
$p_{opt} \leftarrow p_{ref}$;
for $k = 1 : L$ **do**
  /* Neighbour point p_{new} generation */
  for $i = 1 : 3$ **do**
    $p_{new}[i] \leftarrow p_{ref}[i] + rand(-3, 3)$;
  end
  $p_{new}[4] \leftarrow p_{ref}[i] + randint(-4, 4)$;
  /* Selection */
  if $f(p_{new}) < f(p_{opt})$ **then**
    $p_{opt} \leftarrow p_{new}$;
  end
  if $f(p_{new}) < f(p_{ref}) or rand(0, 1) < e^{\frac{-(f(p_{new}) - f(p_{ref}))}{T_k}}$ **then**
    $p_{ref} \leftarrow p_{new}$;
  end
  /* Cooling */
  if $k\%(L/25) == 0$ **then**
    $T_k \leftarrow T_k \cdot cool_{ratio}$;
  end
end

it should be between 0 and 1 as stated by [53].

$$\begin{aligned} T_f &= \frac{-1}{ln(0.05)} \\ T_f &= 0.3333 \\ T_f &= T_0 \cdot cool_{ratio}^{25} \\ cool_{ratio} &= \sqrt[25]{\frac{0.3333}{19.5}} \\ cool_{ratio} &= 0.85 \end{aligned} \tag{17}$$

Now we only have the maximum iterations L as an unknown parameter for which we will later make experiments to determine its best value. The steps of Algorithm 3 are inspired by the work of [54].

VI. RESULTS AND DISCUSSION

A. MODELS

The following models are VRML files and are virtual representations scenarios that could exist on real life chosen so that they had an increasing difficulty among them. They are built with triangular meshes and as a single surface instead of an actual solid model as we only want to include the surface that would be sensed by the robot, otherwise, all the fillings of the solid would be unreachable to the sensor and our model would have a lot of unseen area.

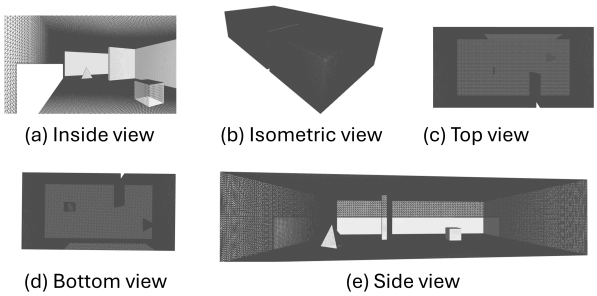

(a) Inside view (b) Isometric view (c) Top view
(d) Bottom view (e) Side view

FIGURE 5. Room with objects model views.

1) ROOM WITH OBJECTS MODEL

This first model is a simple room with one wall that divides it partially into two sections. To increase a little the complexity we decided to add a cube on one section and one pyramid on the other section. A window hole was added to see the interior due to the limitations of our simulator not giving us the ability to look inside the model. Its dimensions are 6 m x 2.7 m x 12 m with a total surface area of 246.634048 m^2. Multiple angles of this model can be seen on Fig. 5 including an inside view.

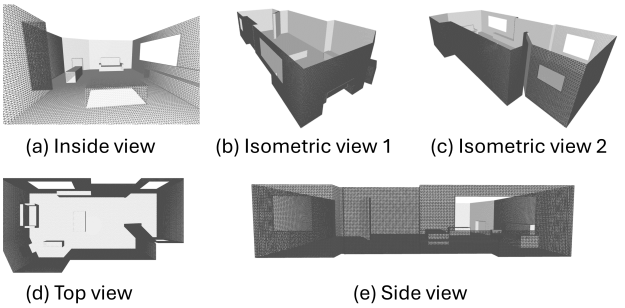


FIGURE 6. Study room model views.

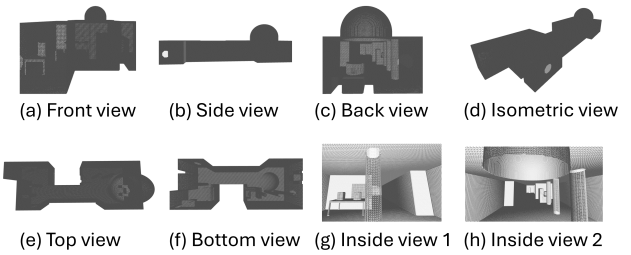


FIGURE 7. Laboratory model views.

2) STUDY ROOM MODEL

This model represents a study room with windows, a chimney on a corner and multiple furniture. We consider this model to have an intermediate complexity due to its general shape and the furniture introduced, specially the sofa. The reason why it is intermediate and not extremely complex is because the desks from the furniture are oversimplified into rectangular prisms. As we have more interior detail it was decided to built the model without a roof to be able to see the results on a better way. Its dimensions are 6 m x 2.7 m x 12.096 m with a total surface area of 170.448196 m^2. Multiple angles of this model can be seen on Fig. 6 including an inside view on Fig. 6(a) where we see with more detail the sofa and the simplified desks.

3) LABORATORY MODEL

This model developed by [55] is a large building containing two sections, one containing a dome plus three columns and two furniture, whereas the other section is more simple containing just one furniture and one column although it has more narrow areas. Both sections are joined with a corridor in the middle of the model. Its dimensions are 7 m x 5.3 m x 20 m with a total surface area of 443.205414 m^2. Multiple angles of this model can be seen on Fig. 7 including two inside views. Fig. 7(g) corresponds to the section where one column and one furniture can be seen, whereas Fig. 7(h) corresponds to the other section where the dome, three columns and two furniture can be seen. We consider this model as the most complex one. Therefore, it will be our base model for the initial experiments as we consider that if the algorithm is able to retrieve this model it will recover all the other models. Still, on the final section once all algorithms have been tuned correctly we will test them on all the models.

B. SETUP PARAMETERS

For all the upcoming experiments we will use the same simulated sensor and robot dimensions shown on Table 2 to have equal conditions between each experiment.

TABLE 2. Setup parameters.

Parameter	Value
Sensor characteristics	
Sensor view angle	80°
Minimum sensor distance	0.5 m
Maximum sensor distance	30 m
Sensor noise deviation	0.012
Sensor horizontal aperture	135°
Sensor vertical aperture	70°
Sensor horizontal points	350
Sensor vertical points	80
Robot dimensions	
Minimum sensor height	0.5 m
Maximum sensor height	2.0 m
Sensor safe distance	0.3 m
Voxel map parameters	
Voxel size	0.15 m

C. OPTIMIZATION METHODS TUNING

Table 3 describes the conditions that will apply to the following experiments where the tuning of each optimization method will be made, we introduce the view numbers which are the total number of iterations of the whole NBV process (not to be confused with the iterations of each optimization method). This set of experiments will be on each method to find their best parameters or modifications, with all of them having the laboratory model as their environment due to its high complexity, which can be seen on Fig. 7.

TABLE 3. Optimization methods tuning simulation conditions.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	100
Optimization method	variable
Objective function	
Total objective function	Equation 1
Area factor	Equation 7
Distance factor	Equation 11
Energy factor	Equation 15

1) NELDER-MEAD

In this subsection we will keep that two-phase method of [6] as an experiment and include another one in which no exhaustive search is used. We will assign the ID of "1.1" to the experiment with exhaustive search and "1.2" for the experiment without it.

The results are shown on Table 5 where we include the previously explained distance d and energy e on a cumulative way by adding the values of all the views, plus a new variable $retr$ for the percentage of area retrieved, computed

TABLE 4. Nelder-Mead experiment variable parameters.

ID	Exhaustive search
1.1	yes
1.2	no

TABLE 5. Nelder-Mead experiment results.

ID	Accumulated d (m)	Accumulated e	$retr(\%)$	$time$ (s)
1.1	353.478	20.044	88.28	3594.35
1.2	327.35	19.116	89.51	297.067

TABLE 6. Nelder-Mead experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
1.1	7.981	4.856	1.374	4.209	1109.946	**1114.155**
1.2	0.000	0.000	0.000	0.000	0.000	**0.000**

as $retr = 100 \cdot nm_{occupied} \cdot (triangles_{retr}/triangles_{total})$ where $triangles_{retr}$ is the number of retrieved triangles which are determined by their overlapping with the occupied voxels obtained from the sensing, and $triangles_{total}$ is the total number of triangles in the VRML file and is known at the beginning of the process.

Seems obvious that the best experiment is "1.2" as it has the best $area_{ret}$ and computational time. However, we will still make a deeper comparison including ranks. We will have two types of rankings, decreasing ranking $ranking_{decr}$ from Equation 19 (the lower the variable the better), and increasing ranking $ranking_{incr}$ from Equation 18 (the higher the variable the better), where $value_{max}$ and $value_{min}$ are the maximum and minimum values of the analyzed variable, and $value_{rank}$ is the current value that will be ranked.

$$ranking_{incr} = 100 \cdot \left| \frac{value_{rank} - value_{max}}{value_{max}} \right| \quad (18)$$

$$ranking_{decr} = 100 \cdot \left| \frac{value_{rank} - value_{min}}{value_{min}} \right| \quad (19)$$

The ranks that will have $ranking_{decr}$ are the distance d_{rank} for accumulated d, energy e_{rank} for accumulated e. $retr_{rank}$ for $retr$ will have $ranking_{incr}$.

A global rank for the objective function which assigns weights for each factor will be computed as:

$$f_{rank} = 0.5retr_{rank} + 0.5(0.7d_{rank} + 0.3e_{rank}) \quad (20)$$

and the total rank $total_{rank}$ will add the function rank plus a new time rank $time_{rank}$ that will be in the form of $ranking_{decr}$ being computed as $total_{rank} = f_{rank} + time_{rank}$. This ranking technique will be the one used on all optimization method experiments as it provides a balance of the objective function and computational time which are the target and main elements of comparison among all optimization methods. The ranked results are shown on Table 6.

We confirm that the best experiment is indeed "1.2" as it has the lowest total rank, and we notice that all its ranks are 0.000 meaning that it is the best experiment in each criterion.

TABLE 7. (1+1)-ES $\sigma_{initial}$ experiment variable parameters.

ID	c	gen_{max}	$\sigma_{initial}$
2.1	0.817	500	1
2.2	0.817	500	5
2.3	0.817	500	7.5
2.4	0.817	500	10

TABLE 8. (1+1)-ES $\sigma_{initial}$ experiment results.

ID	Accumulated d (m)	Accumulated e	$retr(\%)$	$time$ (s)
2.1	253.761	14.506	56.42	913.133
2.2	255.649	17.796	83.06	508.707
2.3	331.917	21.595	84.55	435.707
2.4	283.684	18.766	85.03	378.907

TABLE 9. (1+1)-ES $\sigma_{initial}$ experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
2.2	0.000	0.000	2.317	1.158	34.256	35.415
2.3	29.833	21.346	0.565	13.926	14.990	28.916
2.4	10.966	5.453	0.000	4.656	0.000	4.656

This gets us to the conclusion that exhaustive search for orientation on this specific problem is not necessary and it has a negative impact where the increase on the computational time is the most drastic. We should use the pure Nelder-Mead algorithm without exhaustive search.

2) (1+1)-ES

The first set of experiments of this algorithm will be for obtaining the best value of the parameter $\sigma_{initial}$ with $gen_{max} = 500$ as discussed before. A common value of $\sigma_{initial}$ is 1 although we have the hypothesis that it is small compared to the range of our variables and it may provide a small change making the algorithm explore less possibilities, that is why we will propose bigger numbers on other experiments starting on 5 with 2.5 steps until 10. As we now have more parameters, we will create an experimental matrix shown on Table 7 with their corresponding parameters.

The results of each experiment are shown on Table 8, where we immediately notice that experiment "2.1" has an extremely low $retr$, we will not consider it an outlier and will be omitted on the ranked results shown on Table 9 as it could affect the rest of the ranks due to them being relative.

The experiment with the smaller total rank is "2.4", meaning that the best value for $\sigma_{initial}$ is 10. Choosing that parameter value we end up with a total time of 378.9s which can be further reduced by modifying the value of gen_{max} that currently is the only stopping mechanism. We will make an analysis by repeating the experiment "2.4" that had $gen_{max} = 500$ and store information of the first generation gen that has a relative error of 10% or less compared to the actual value at $gen = 500$. The information stored will be the value of such gen with its corresponding values of m and σ at that specific generation as we want to see which of those

TABLE 10. (1+1)-ES stopping mechanism analysis.

	gen	*m*	σ
Upper limit	442	15	4.455
Lower limit	88	1	0.117
Range	354	14	4.338
Mean	307.277	4.877	0.748
Standard deviation	86.605	3.389	0.842
Coefficient of variation (%)	28.185	69.488	112.556

TABLE 11. (1+1)-ES gen_{max} experiment variable parameters.

ID	*c*	gen_{max}	$\sigma_{initial}$
2.4.1	0.817	500	10
2.4.2	0.817	308	10

TABLE 12. (1+1)-ES gen_{max} experiment results.

ID	Accumulated *d* (*m*)	Accumulated *e*	*retr* (%)	*time* (s)
2.4.1	283.684	18.766	85.03	378.907
2.4.2	272.077	16.544	87.78	120.493

TABLE 13. (1+1)-ES gen_{max} experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
2.4.1	4.266	13.435	3.133	5.075	214.463	219.538
2.4.2	0.000	0.000	0.000	0.000	0.000	0.000

three variables represents a better stopping mechanism due to them being the only changing variables from the algorithm.

Table 10 shows a summary of the stored results from all 100 views where we have their limits and range plus their mean and standard deviation in order to compute the coefficient of variation as (100 · standard deviation)/mean, that will help us to decide how constant each variable is.

The most constant variable is *gen* which has the smallest coefficient of variation. Although 28.18% may seem as a big coefficient of variation, it still falls under the acceptable range which is up to 30% as explained on [56]. With this results, we get to the conclusion that no other variable is acceptable as a stopping mechanism, it will still be gen_{max} although now it can be reduced up to $gen_{max} = 308$ which is the mean of *gen* rounded to the next integer.

To test that by reducing gen_{max} the time is reduced and the final result has not been compromised we will compare it to the original experiment. Table 11 shows both experiments with their corresponding gen_{max}.

The results of each experiment are shown on Table 12 where we can see that the total time is indeed reduced on "2.4.2". Still, we will rank both experiments on Table 13 using the same ranking techniques as before to make sure that the global performance is not affected.

With this results we can confirm that "2.4.2" is the best experiment, as having $gen_{max} = 308$ reduces the computational time and it even upgrades the distance, energy, and percentage retrieved. This upgrade in all the other factors

TABLE 14. Simulated annealing experiment variable parameters.

ID	T_0	$cool_{ratio}$	*L*
3.1	19.5	0.85	100
3.2	19.5	0.85	200
3.3	19.5	0.85	300
3.4	19.5	0.85	400
3.5	19.5	0.85	500

TABLE 15. Simulated annealing experiment results.

ID	Accumulated *d* (*m*)	Accumulated *e*	*retr* (%)	*time* (s)
3.1	236.371	18.628	77.91	54.333
3.2	272.426	23.732	81.04	133.413
3.3	317.389	22.772	87.69	176.307
3.4	311.744	28.827	89.38	228.68
3.5	389.03	22.685	89.43	254.667

TABLE 16. Simulated annealing experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
3.3	1.811	0.380	1.946	1.664	0.000	1.664
3.4	0.000	27.071	0.056	4.089	29.706	33.794
3.5	24.791	0.000	0.000	8.677	44.445	53.122

may seem illogical as we cannot get a better value for the objective function with less generations and in fact we did not, the results from the objective function were worst on some iterations, but our upgrade is not on individual views but on the final result. This could be caused by the sensor falling on a worst view on some iteration and therefore the conditions for the future iterations change completely making the robot follow a different path, causing this downgrade on a certain iteration to cause a future upgrade on the global result. This may not always happen, but the important thing is that we did not see any substantial negative impact and we will take the little upgrade as circumstantial but the big impact on the time is indeed considerable. Our conclusion regarding this algorithm is that the best parameters should be $gen_{max} = 308$ and $\sigma_{inital} = 10$.

3) SIMULATED ANNEALING

This last method requires only one parameter variation which will be *L*. Table 14 shows all experiments with their parameters, where we keep the values of T_0 and $cool_{ratio}$ constant with *L* being the only changing parameter. As it refers to the number of iterations, we will go for change of 100 within each experiment starting at 100 up to 500, exceeding this last value would cause the algorithm to have a significant execution time which we desire to decrease.

The results of each experiment are shown on Table 15 where we notice that both the *retr* and *time* increase when *L* increases. We will filter the experiments so than only the ones with *retr* greater than %85 are kept as the others are considered outliers, and we will apply the same ranking techniques as before only for the filtered experiments. The ranked results are shown on Table 16.

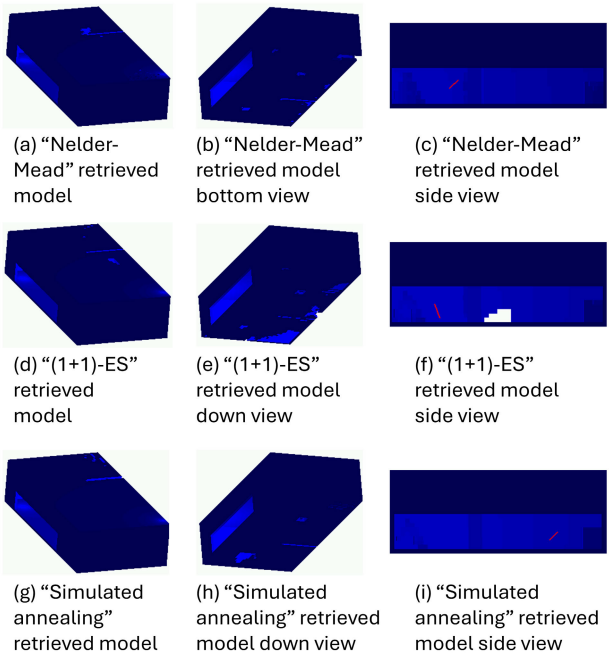


FIGURE 8. Room with objects model experiment retrieved models.

TABLE 17. Room with objects model experiment simulation parameters.

Parameter	Value
General considerations	
Environment	room with objects model
Number of views	50
Optimization method	variable
Objective function	
Total objective function	Equation 1
Area factor	Equation 7
Distance factor	Equation 11
Energy factor	Equation 15

We confirm that "3.3" is the best experiment with lowest total rank, meaning that the best value for the parameter L is 300, even if on other experiments *retr* was greater but the downgrade on all the other factors including time was worst.

D. FINAL COMPARISONS

Now that we tuned our optimization methods with their best parameters, we will test this three methods on all of our models.

1) ROOM WITH OBJECTS MODEL COMPARISONS

This first environment is the model from Fig. 5. The simulation parameters for this experiments are shown on Table 17 and we will choose the number of views as 50 because it is a relatively simple model. Still, it was tested with 200 views and we noticed that the improvement on the retrieved model was negligible so we kept the experiments with 50 views.

On Fig. 8 we can see the retrieved area from each experiment where we only show the occupied voxels in blue.

TABLE 18. Room with objects model experiment results.

Method	Accumulated d (m)	Accumulated e	$retr\,(\%)$	$time$ (s)
Nelder-Mead	98.435	5.230	83.47	131.053
(1+1)-ES	119.858	6.013	84.03	34.733
Simulated annealing	146.451	6.218	83.54	35.72

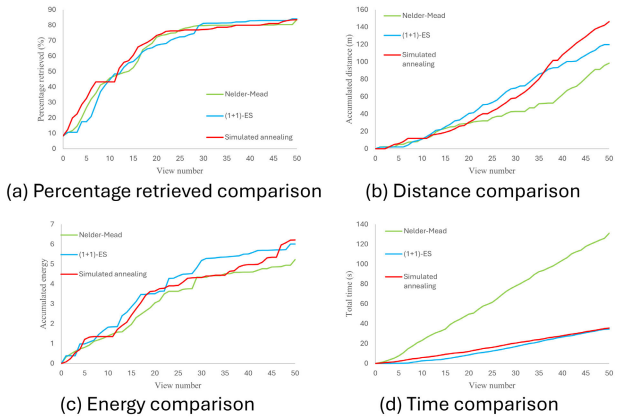


FIGURE 9. Room with objects model experiment graphs.

Fig. 8(a), 8(d) and 8(g) show us the roof and we can see that there are some differences but in general we can say the three methods obtain similar results. On Fig. 8(b), 8(e) and 8(h) we can see the bottom of the model and clearly the (1+1)-ES method represented the worst recovery of the floor. Finally, on Fig. 8(c), 8(f) and 8(i) a side view is shown where we can see some of the interior recovery, and on all three experiments the wall, cube and pyramid were retrieved successfully although the pyramid seems to be pixelated but this is due to the voxel size. One last observation in these side views is that on the (1+1)-ES experiment it seems that a little section of a wall was not recovered.

The results for each method can be seen on Table 18 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods and the (1+1)-ES method has the highest % retrieved.

On Fig. 9(a) we can see that the *retr* is very close among the three methods an that on different view numbers the best and worst method were changing, so on this variable there is no clear winner but all of them seem to behave correctly. On Fig. 9(b) the Nelder-Mead experiment was clearly the best as from view #20 it was the one with the smallest distance, plus it was also the case on the energy comparison on Fig. 9(c). Finally, on Fig. 9(d) we can see the total time and the Nelder-Mead method is the worst one with the biggest time on all the views by a huge difference, where (1+1)-ES and simulated annealing seem to be tied on all the view numbers and we know from the tables that the best was the (1+1)-ES experiment but on the plot that small difference is not noticeable.

TABLE 19. Room with objects model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
Nelder-Mead	0.000	0.000	0.666	0.333	277.313	277.646
(1+1)-ES	21.764	14.952	0.000	9.860	0.000	9.860
Simulated annealing	48.779	18.875	0.583	20.196	2.841	23.036

TABLE 20. Study room model experiment simulation parameters.

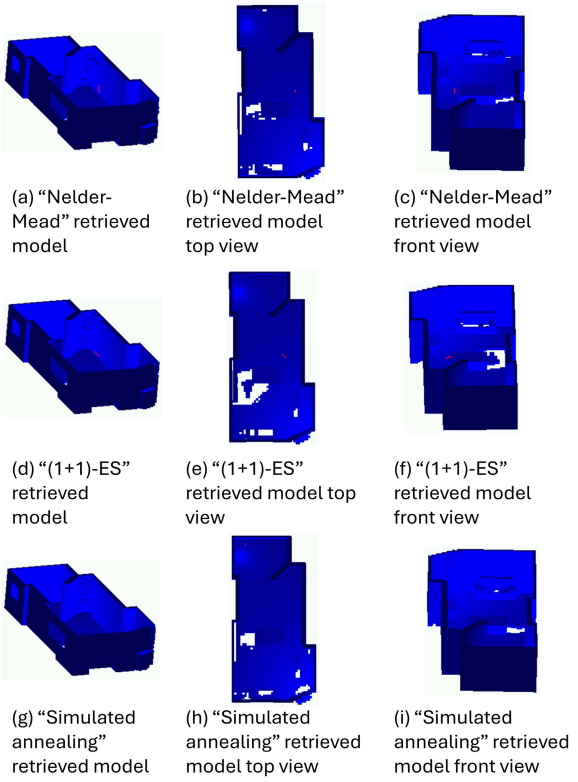
Parameter	Value
General considerations	
Environment	study room model
Number of views	100
Optimization method	variable
Objective function	
Total objective function	Equation 1
Area factor	Equation 7
Distance factor	Equation 11
Energy factor	Equation 15

As we established that the decision should be made considering all the factors of the objective function and the total time, we will use the same ranking technique from previous experiments and can be seen on Table 19. With this criteria, the best method on this environment is the (1+1)-ES algorithm and it was the best on all the individual criteria except for the distance and energy, meaning that it uses more resources, but the balance with the other criteria means that it is worth it. Overall, all three methods provide good results with little differences between them except for the huge difference in time of the Nelder-Mead method.

2) STUDY ROOM MODEL COMPARISONS

This second environment is the study room which can be seen on Fig. 6. The simulation parameters for this experiments are shown on Table 20 and we increase the number of views to 100 as this model is more complex than the previous one. Still, it was tested with 50 views and the difference on the retrieved models is indeed significant. Then, it was tested with 200 views and we noticed that the improvement on the retrieved model was now negligible so we kept the experiments with 100 views.

On Fig. 10(a), 10(d) and 10(g) we can see that there are no noticeable differences on the general shape of the model. However, on Fig. 10(b), 10(e) and 10(h) we can see the bottom of the models is clearly where the missing % retrieved is as they have a lot of holes, with the (1+1)-ES experiment having the biggest holes and the other two methods being similar between them. Finally, on On Fig. 10(c), 10(f) and 10(i) we have a view with more interior detail to see the furniture and we notice that the sofa in front of the top wall is the one with more issues as it has holes although the simulated annealing one seems to be the best regarding the sofa as it only has a tiny hole on the middle of the sofa while the others have bigger holes.



(a) "Nelder-Mead" retrieved model
(b) "Nelder-Mead" retrieved model top view
(c) "Nelder-Mead" retrieved model front view
(d) "(1+1)-ES" retrieved model
(e) "(1+1)-ES" retrieved model top view
(f) "(1+1)-ES" retrieved model front view
(g) "Simulated annealing" retrieved model
(h) "Simulated annealing" retrieved model top view
(i) "Simulated annealing" retrieved model front view

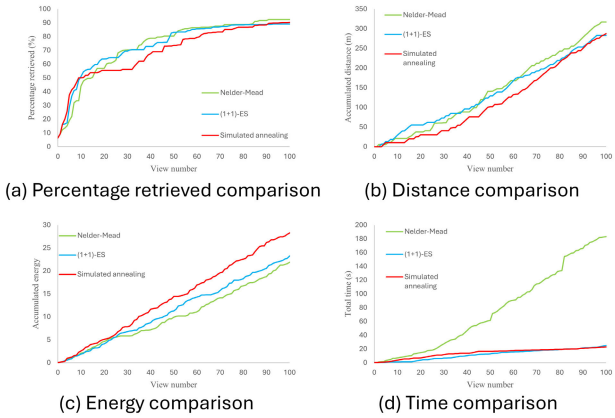
FIGURE 10. Study room model experiment retrieved models.

TABLE 21. Study room model experiment results.

Method	Accumulated d (m)	Accumulated e	$retr(\%)$	$time$ (s)
Nelder-Mead	316.973	21.902	92.41	183.333
(1+1)-ES	283.049	23.260	89.17	25.027
Simulated annealing	287.862	28.270	90.26	22.813

The results for each method can be seen on Table 21 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods besides it having slightly more *retr*.

On Fig. 11(a) we can see that the simulated annealing method provided less *retr* on most of the views although at the end all three methods seem to have similar results. On Fig. 11(b) the simulated annealing algorithm was the one that needed less distance on most of the views although at the end the (1+1)-ES won by having the smallest distance on the last view. On Fig. 11(c) it is quite the opposite as the Nelder-Mead experiment which was the one with the highest distance now is the one with the smallest energy consumption on all the views after view #25 approximately. Finally, on Fig. 11(d) we can see the total time and the Nelder-Mead method is the worst one with the biggest time on all the views by a huge difference, where (1+1)-ES and simulated annealing seem to be tied on all the view numbers until the last ones where the simulated annealing



(a) Percentage retrieved comparison (b) Distance comparison

(c) Energy comparison (d) Time comparison

FIGURE 11. Study room model experiment graphs.

TABLE 22. Study room model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
Nelder-Mead	11.985	0.000	0.000	4.195	703.624	707.818
(1+1)-ES	0.000	6.200	3.506	2.683	9.702	12.385
Simulated annealing	1.700	29.073	2.327	6.119	0.000	6.119

wins with the smallest time although it is by a little difference.

The results after the ranking technique is applied can be seen on Table 22. With this criteria, the best method on this environment is the simulated annealing even if it was the best only in the time rank. This means that even if it was not the best on the other individual criteria, it had more balance than the other methods. Overall, all three methods provide good results with little differences between them except for the huge difference in time of the Nelder-Mead method.

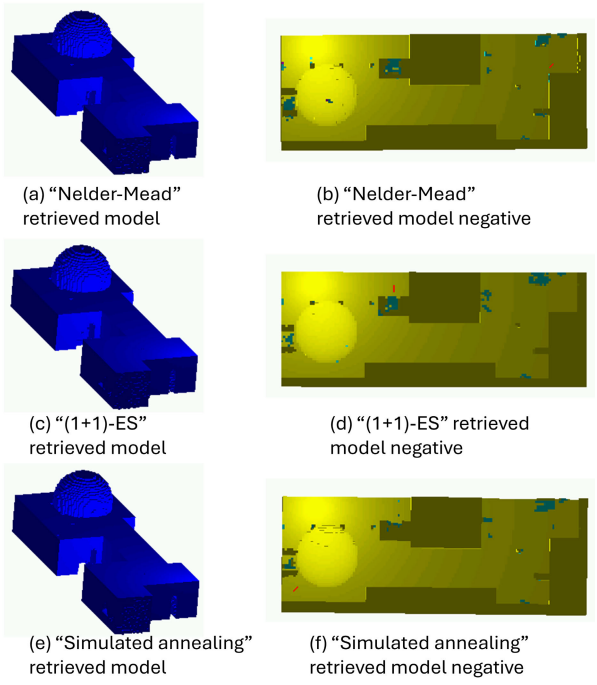
3) LABORATORY MODEL COMPARISONS

This last environment is the laboratory model which can be seen on Fig. 7 and is the model used to do the tuning. We will collect the best versions of the three methods on a single experimental matrix for a more straight forward comparison and we will provide new graphs of their performance. The simulation parameters for this experiments are shown on Table 23 and we keep the number of views as 100 which was used on the previous experiments mainly because we consider this environment a complex one and we want to provide the algorithm enough opportunity to retrieve the model. Still, it was tested with 200 views and we noticed that the improvement on the retrieved model was negligible so we kept the experiments with 100 views.

On Fig. 12 we present the retrieved models and their negative that consists of all the voxels that are not yet defined as occupied or empty, which are occluded voxels in yellow, occplane voxels on cyan and unmarked voxels on purple. Notice that visually the difference is almost unnoticeable although the (1+1)-ES experiment seems to have a few more

TABLE 23. Laboratory model experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	100
Optimization method	variable
Objective function	
Total objective function	Equation 1
Area factor	Equation 7
Distance factor	Equation 11
Energy factor	Equation 15



(a) "Nelder-Mead" retrieved model (b) "Nelder-Mead" retrieved model negative

(c) "(1+1)-ES" retrieved model (d) "(1+1)-ES" retrieved model negative

(e) "Simulated annealing" retrieved model (f) "Simulated annealing" retrieved model negative

FIGURE 12. Laboratory model experiment retrieved models.

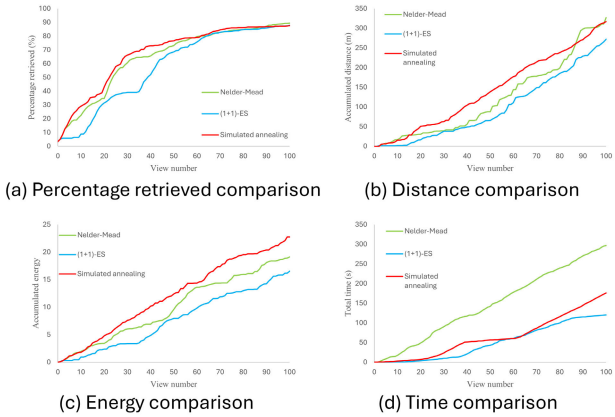
TABLE 24. Laboratory model experiment results.

Method	Accumulated d (m)	Accumulated e	$retr\,(\%)$	$time$ (s)
Nelder-Mead	327.351	19.118	89.51	297.067
(1+1)-ES	272.077	16.544	87.78	120.493
Simulated annealing	317.389	22.772	87.69	176.307

occplane voxels but we will rely more on the numerical information.

The results for each method can be seen on Table 24 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods but having the most percentage retrieved.

On Fig. 13(a) we can see that the simulated annealing method provided more *retr* on most of the views although at the end in the last views the best method turned out to be Nelder-Mead by a little difference. On Fig. 13(b) the (1+1)-ES algorithm was the one that needed less distance from all methods on all the views and is the same case for the



(a) Percentage retrieved comparison (b) Distance comparison

(c) Energy comparison (d) Time comparison

FIGURE 13. Laboratory model experiment graphs.

TABLE 25. Laboratory model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$\boldsymbol{total_{rank}}$
Nelder-Mead	20.316	15.546	0.000	9.442	146.542	155.984
(1+1)-ES	0.000	0.000	1.933	0.966	0.000	0.966
Simulated annealing	16.654	37.644	2.033	12.492	46.321	58.813

energy on Fig. 13(c). On Fig. 13(e) we can see the total time and the Nelder-Mead method is the one with the biggest time on all the views, where (1+1)-ES and simulated annealing seem to be tied on the first views and around view 60 although at the end (1+1)-ES wins having a considerable less time around the last views.

The ranked results are shown on Table 25. With this criteria, the best method on this environment is the (1+1)-ES algorithm and it was the best on all the individual criteria except for the percentage retrieved but the difference was not considerable. Overall, all three methods provide good results with little differences between them except for the difference in time of the Nelder-Mead method although it is less extreme than on the previous two models.

VII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

One crucial part was defining the backstepping algorithm, as we noticed that without them the robot was getting stuck on local minima and the process ended with poor results regarding the retrievement of the model. The reason why we chose to deal with this local minima problem by adding an external algorithm such as our backstepping technique is because if we wanted to deal with it purely with the optimization method, we would need to choose either a full exhaustive search which would not be feasible due to its immense possible points, or to use some optimization method designed for escaping local minima like tabu search, but on previous experiments we have made beyond the scope of this paper we noticed that even with such type of methods our backstepping algorithm was still being used making our local minima problem extreme enough to be handled without this technique.

As for the optimization methods tested, an adequate tuning of their individual parameters was done at the best of our abilities because if we decided to make a full experimental matrix it would not been optimal due to the nature of the parameters being continuous and the endless combinations of all of them. Even is no full experimental matrix was made, we still got the opportunity to notice that by changing the parameters we could end up with huge differences in our results. On the Nelder-Mead we had no parameters but instead we chose to compare two versions of the method, but on the (1+1)-ES we now included parameter variations and we managed to upgrade the $total_{rank}$ when tuning its first parameter from 35.415 being the highest to 4.656 being the lowest and the best, and then from 219.538 to even 0.000 on its second parameter meaning that the final version was the best in all the criteria. For the simulated annealing we only had one parameter tuning where we got to improve the $total_{rank}$ from 53.122 being the highest to 1.664 being the best. Our first hypothesis that the changes in the parameters when tuning each optimization method have an impact on their results is correct.

During the final comparison we noticed that even if on two models the best algorithm was the (1+1)-ES, on the remaining model the simulated annealing was the best, this means that there is no clear superior algorithm between those two for all possible environments but it varies depending on the scene, although the overall results on both of them were similar so there should be no problem on using either of those two methods on any environment. The main observation was with the Nelder-Mead method as it was the worst on all three models on its $total_{rank}$ with a significant difference from the other two methods, with the most affected criterion being its high computational time even if it was considerably reduced against its two-phase variant method, but still not enough to compete against (1+1)-ES and simulated annealing on that aspect even if it had overall good results on all the other criteria. Our second and final hypothesis that we can obtain significant differences among our proposed methods is correct when comparing (1+1)-ES and simulated annealing against the Nelder-Mead method which we consider as the worst.

B. FUTURE WORK

One main assumption of this work was the simplification of the movement of the robot as it was decided that due to being on a simulation the sensor could move from one point to another without any problem by just checking if the space around it was empty. However, it would be ideal that a deeper analysis on the robot kinematics and path trajectory was done on a navigation module to make easier its future implementation on a real robot inside a real environment.

Finally, during the final experiments done on all three environments, we noticed that the one with an overall higher percentage of retrievement was the study room. The problem

we identified was that the remaining unretrieved area was concentrated on big holes in the floor, making those missing parts more noticeable, and the other models that were lower in terms of their retrievement percentage presented less incomplete sections at least visually as the missing parts were more scattered. For this, we suggest to modify the area factor from the objective function so that even if two candidate images have the same values on their individual area proportions, different weights or priorities are assigned if they are concentrated or scattered. This is only a proposal but a deeper analysis and experiments should be made to test if indeed it provides better results or not.

ACKNOWLEDGMENT

Everardo Shain-Ruvalcaba thanks the institutions Tecnológico de Monterrey and Consejo Nacional de Humanidades, Ciencias y Tecnologías (CONAHCYT) for making this project possible with their support.

REFERENCES

[1] Y. Zhang, "Research into the engineering application of reverse engineering technology," *J. Mater. Process. Technol.*, vol. 139, nos. 1–3, pp. 472–475, Aug. 2003.
[2] J. Wang, Z. Li, W. Hu, Y. Shao, L. Wang, R. Wu, K. Ma, D. Zou, and Y. Chen, "Virtual reality and integrated crime scene scanning for immersive and heterogeneous crime scene reconstruction," *Forensic Sci. Int.*, vol. 303, Oct. 2019, Art. no. 109943.
[3] D. Filko, D. Marijanović, and E. K. Nyarko, "Automatic robot-driven 3D reconstruction system for chronic wounds," *Sensors*, vol. 21, no. 24, p. 8308, Dec. 2021.
[4] C. McGreavy, L. Kunze, and N. Hawes, "Next best view planning for object recognition in mobile robotics," in *Proc. CEUR Workshop*, vol. 1782, 2017, pp. 1–9.
[5] M. Naazare, F. G. Rosas, and D. Schulz, "Online next-best-view planner for 3D-exploration and inspection with a mobile manipulator robot," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 3779–3786, Apr. 2022.
[6] J. M. Sanchiz and R. B. Fisher, "A next-best-view algorithm for 3D scene recovery with 5 degreesof freedom," in *Proc. Brit. Mach. Vis. Conf.*, 1999, pp. 1–10.
[7] M. L. Albalate, M. Devy, J. Miguel, and S. Marti, "Perception planning for an exploration task of a 3D environment," in *Proc. Int. Conf. Pattern Recognit.*, vol. 3, Jun. 2002, pp. 704–707.
[8] R. Pito, "A sensor-based solution to the 'next best view' problem," in *Proc. 13th Int. Conf. Pattern Recognit.*, vol. 1, Jun. 1996, pp. 941–945.
[9] T. Olsson, "Vision guided force control in robotics," M.S. thesis, Dept. Autom. Control, Lund Inst. Technol., Lund, Sweden, 2001.
[10] L. Torabi and K. Gupta, "An autonomous six-DOF eye-in-hand system for in situ 3D object modeling," *Int. J. Robot. Res.*, vol. 31, no. 1, pp. 82–100, Jan. 2012.
[11] T. Foissotte, O. Stasse, A. Escande, and A. Kheddar, "A next-best-view algorithm for autonomous 3D object modeling by a humanoid robot," in *Proc. Humanoids 8th IEEE-RAS Int. Conf. Humanoid Robots*, Dec. 2008, pp. 333–338.
[12] R. Team, "HRP-2," Dec. 2023. IEEE Spectrum, *HRP-2*. Accessed: Dec. 2023. [Online]. Available: https://robotsguide.com/robots/hrp2
[13] R. Ashour, T. Taha, J. M. M. Dias, L. Seneviratne, and N. Almoosa, "Exploration for object mapping guided by environmental semantics using UAVs," *Remote Sens.*, vol. 12, no. 5, p. 891, Mar. 2020.
[14] G. Hardouin, J. Moras, F. Morbidi, J. Marzat, and E. M. Mouaddib, "Next-best-view planning for surface reconstruction of large-scale 3D environments with multiple UAVs," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 1567–1574.
[15] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon, 'next-best-view' planner for 3D exploration," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Nov. 2016, pp. 1462–1468.
[16] M. Hržica, R. Cupec, and I. Petrović, "Active vision for 3D indoor scene reconstruction using a 3D camera on a pan-tilt mechanism," *Adv. Robot.*, vol. 35, nos. 3–4, pp. 153–167, Feb. 2021.
[17] L. M. Wong, C. Dumont, and M. A. Abidi, "Determining optimal sensor poses in 3D object inspection," in *Proc. Conf. Quality Control Artif. Vis.*, Takamatsu, Japan. Princeton, NJ, USA: Citeseer, 1998, pp. 371–377.
[18] N. A. Massios and R. B. Fisher, *A Best Next View Selection Algorithm Incorporating a Quality Criterion*, vol. 2. Princeton, NJ, USA: Citeseer, 1998.
[19] T. Foissotte, O. Stasse, P.-B. Wieber, and A. Kheddar, "Using NEWUOA to drive the autonomous visual modeling of an object by a humanoid robot," in *Proc. Int. Conf. Inf. Autom.*, Jun. 2009, pp. 78–83.
[20] J. I. Vásquez-Gómez, E. Löpez-Damian, and L. E. Sucar, "View planning for 3D object reconstruction," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2009, pp. 4015–4020.
[21] J. E. Banta, L. R. Wong, C. Dumont, and M. A. Abidi, "A next-best-view system for autonomous 3-D object reconstruction," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 5, pp. 589–598, May 2000.
[22] Y. Kong, F. Zhu, H. Sun, Z. Lin, and Q. Wang, "A generic view planning system based on formal expression of perception tasks," *Entropy*, vol. 24, no. 5, p. 578, Apr. 2022.
[23] J. I. Vasquez-Gomez, D. Troncoso, I. Becerra, E. Sucar, and R. Murrieta-Cid, "Next-best-view regression using a 3D convolutional neural network," *Mach. Vis. Appl.*, vol. 32, no. 2, pp. 1–14, Mar. 2021.
[24] R. Almadhoun, A. Abduldayem, T. Taha, L. Seneviratne, and Y. Zweiri, "Guided next best view for 3D reconstruction of large complex structures," *Remote Sens.*, vol. 11, no. 20, p. 2440, Oct. 2019.
[25] C. Connolly, "The determination of next best views," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 2, Sep. 1985, pp. 432–435.
[26] F. Bissmarck, M. Svensson, and G. Tolt, "Efficient algorithms for next best view evaluation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2015, pp. 5876–5883.
[27] C. Potthast and G. S. Sukhatme, "A probabilistic framework for next best view estimation in a cluttered environment," *J. Vis. Commun. Image Represent.*, vol. 25, no. 1, pp. 148–164, Jan. 2014.
[28] J. I. Vasquez-Gomez, L. E. Sucar, and R. Murrieta-Cid, "Hierarchical ray tracing for fast volumetric next-best-view planning," in *Proc. Int. Conf. Comput. Robot Vis.*, May 2013, pp. 181–187.
[29] J. I. Vasquez-Gomez, L. E. Sucar, R. Murrieta-Cid, and J.-C. Herrera-Lozada, "Tree-based search of the next best view/state for three-dimensional object reconstruction," *Int. J. Adv. Robotic Syst.*, vol. 15, no. 1, Jan. 2018, Art. no. 1729881418754575.
[30] L. M. González-Desantos, J. Martínez-Sánchez, H. González-Jorge, L. Díaz-Vilariño, and B. Riveiro, "New discretization method applied to NBV problem: Semioctree," *PLoS ONE*, vol. 13, no. 11, Nov. 2018, Art. no. e0206259.
[31] G. Hetzel, B. Leibe, P. Levi, and B. Schiele, "3D object recognition from range images using local feature histograms," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. CVPR*, vol. 2, Jul. 2001, pp. 1–6.
[32] J. I. Vasquez-Gomez, L. E. Sucar, R. Murrieta-Cid, and E. Lopez-Damian, "Volumetric next-best-view planning for 3D object reconstruction with positioning error," *Int. J. Adv. Robotic Syst.*, vol. 11, no. 10, p. 159, Oct. 2014.
[33] M. Steinbrink, "Sampling-based exploration strategies for mobile robot autonomy," Ph.D. dissertation, Fac. Math. Comp. Sci., TU Bergakademie Freiberg, Freiberg, Germany, 2023.
[34] Y. Zhao, Z. Xiong, S. Zhou, J. Wang, L. Zhang, and P. Campoy, "Perception-aware planning for active SLAM in dynamic environments," *Remote Sens.*, vol. 14, no. 11, p. 2584, May 2022.
[35] A. Batinovic, A. Ivanovic, T. Petrovic, and S. Bogdan, "A shadowcasting-based next-best-view planner for autonomous 3D exploration," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2969–2976, Apr. 2022.
[36] H. Zhu, C. Cao, Y. Xia, S. Scherer, J. Zhang, and W. Wang, "DSVP: Dual-stage viewpoint planner for rapid exploration by dynamic expansion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 7623–7630.
[37] S. Karaman and E. Frazzoli, "Incremental sampling-based algorithms for optimal motion planning," in *Robotics: Science and Systems VI*. Cambridge, MA, USA: MIT Press, 2011.
[38] M. Steinbrink, P. Koch, B. Jung, and S. May, "Rapidly-exploring random graph next-best view exploration for ground vehicles," in *Proc. Eur. Conf. Mobile Robots (ECMR)*, Aug. 2021, pp. 1–7.
[39] E. Dunn and J.-M. Frahm, "Next best view planning for active model improvement," in *Proc. BMVC*, Jan. 2009, pp. 1–11.
[40] J. E. Bresenham, "Algorithm for computer control of a digital plotter," in *Seminal Graphics: Pioneering Efforts That Shaped the Field*. New York, NY, USA: Association For Computing Machinery, 1998, pp. 1–6.

[41] A. Fujimoto, T. Tanaka, and K. Iwata, "ARTS: Accelerated ray-tracing system," *IEEE Comput. Graph. Appl.*, vol. CGA-6, no. 4, pp. 16–26, Apr. 1986.
[42] K. Rajwar, K. Deep, and S. Das, "An exhaustive review of the Metaheuristic algorithms for search and optimization: Taxonomy," *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 13187–13257, 2023.
[43] J. A. Nelder and R. Mead, "A simplex method for function minimization," *Comput. J.*, vol. 7, no. 4, pp. 308–313, Jan. 1965.
[44] M. H. Wright, *Direct Search Methods: Once Scorned* (Pitman Research Notes in Mathematics Series). New York, NY, USA: Pitman, 1996, pp. 191–208.
[45] M. J. D. Powell, "Direct search algorithms for optimization calculations," *Acta Numerica*, vol. 7, pp. 287–336, Jan. 1998.
[46] T. G. Kolda, R. M. Lewis, and V. Torczon, "Optimization by direct search: New perspectives on some classical and modern methods," *SIAM Rev.*, vol. 45, no. 3, pp. 385–482, Jan. 2003.
[47] I. Boussaïd, J. Lepagnot, and P. Siarry, "A survey on optimization metaheuristics," *Inf. Sci.*, vol. 237, pp. 82–117, Jul. 2013.
[48] T. Bartz-Beielstein, J. Branke, J. Mehnen, and O. Mersmann, "Evolutionary algorithms," *Wiley Interdiscipl. Rev., Data Mining Knowl. Discovery*, vol. 4, no. 3, pp. 178–195, 2014.
[49] W. T. Vetterling, S. A. Teukolsky, W. H. Press, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge, U.K.: Cambridge Univ. Press, 1999.
[50] C. A. C. Coello and C. S. P. Zacatenco, "Chapter lecture topic, Introducción a la computación evolutiva," Dept. Comput., CINVESTAV-IPN, México, 2004.
[51] H.-P. Schwefel, *Numerical Optimization of Computer Models*. Hoboken, NJ, USA: Wiley, 1981.
[52] H.-P. Schwefel, *Evolution and Optimum Seeking*. Hoboken, NJ, USA: Wiley, 1995.
[53] M.-W. Park and Y.-D. Kim, "A systematic procedure for setting parameters in simulated annealing algorithms," *Comput. Operations Res.*, vol. 25, no. 3, pp. 207–217, Mar. 1998.
[54] D. Henderson, S. H. Jacobson, and A. W. Johnson, "The theory and practice of simulated annealing," in *Handbook of Metaheuristics*. Boston, MA, USA: Springer, 2003, pp. 287–319.
[55] K. Klein and V. Sequeira, "View planning for the 3D modelling of real world scenes," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 2, Sep. 2000, pp. 943–948.
[56] A. Mangrio, M. Asif, E. Ahmed, M. Sabir, T. Khan, and I. Jahangir, "Hydraulic performance evaluation of pressure compensating (PC) emitters and micro-tubing for drip irrigation system," *Sci. Technol. Develop., Islamabad*, vol. 32, no. 4, pp. 290–298, 2013.

EVERARDO SHAIN-RUVALCABA received the B.S. degree in mechatronics engineering from Tecnologico de Monterrey, Toluca, Mexico, in 2022. He is currently pursuing the M.S. degree in engineering with Tecnologico de Monterrey, Mexico City, Mexico. His current research interests include optimization modeling and algorithms.

EFRAÍN LÓPEZ-DAMIAN received the B.S. and M.S. degrees in electronic systems from Tecnologico de Monterrey, in 1997 and 2001, respectively, and the Ph.D. degree from the Institut National des Sciences Appliquées de Toulouse, France. In 2007, he joined the Computer Science Group, INAOEP, Puebla, Mexico, as a Postdoctoral Researcher, working in view planning for 3D modeling. Currently, he is part of the faculty of Tecnológico de Monterrey, a member with the Center for Research in Automotive Mechatronics (CIMA), and a Graduate Coordinator. During the Ph.D. degree, he works part of the European Cogniron Project on grasp planning for 3D arbitrary object manipulation within the Robotics and Artificial Intelligence Group, LAAS-CNRS. His research interests include robot manipulation, human–robot collaboration, and view planning.